

به نام خدا

پروژه درس سیستم عامل – مهسا نوری شیرازی 4021231087

موضوع: پردازش موازی در شبکه عصبی کانولوشنی

نام پروژه: CNN + Threads + Semaphores

لینک مشاهده پیاده سازی پروژه در اپارات:

<https://www.aparat.com/v/zkw0r48>

شبکه‌های عصبی کانولوشنی یا CNN یکی از مهم‌ترین معماری‌های یادگیری عمیق در حوزه‌هایی مانند بینایی ماشین، تشخیص الگو، پردازش تصویر و تشخیص ارقام دست‌نویس هستند.

هدف این پروژه، پیاده‌سازی یک شبکه CNN ساده برای تشخیص ارقام دیتاست MNIST و هم‌زمان به‌کارگیری مفاهیم رشته‌ها (Threads) و سمافورها (Semaphores) در مدیریت اجرای مراحل مختلف شبکه بوده است.

در این پروژه، مدل ابتدا روی دیتاست MNIST آموزش داده می‌شود و سپس کاربر می‌تواند یک رقم را به صورت دستی در رابط گرافیکی رسم کند تا مدل آن را پیش‌بینی کند. همچنین برای رعایت خواسته‌ی پروژه، اجرای لایه‌ها و epochها به صورت جداگانه و هماهنگ‌شده با thread و semaphore انجام می‌شود.

هدف‌های اصلی این پروژه عبارت‌اند از:

1. آشنایی با ساختار شبکه‌های CNN
2. پیاده‌سازی مدل تشخیص ارقام دست‌نویس با استفاده از TensorFlow/Keras
3. استفاده از Thread برای اجرای موازی مراحل شبکه و epochها
4. استفاده از Semaphore برای کنترل دسترسی به داده‌های اشتراکی
5. ساخت یک رابط گرافیکی ساده برای نمایش نتیجه آموزش و پیش‌بینی
6. آزمایش و اجرای پروژه در محیط WSL / Ubuntu

دیتاست MNIST شامل 60,000 نمونه آموزشی و 10,000 نمونه آزمایشی از ارقام دست‌نویس 0 تا 9 است.

اندازه‌ی هر تصویر 28×28 پیکسل بوده و به صورت خاکستری ذخیره شده است.

در این پروژه، تصاویر ابتدا نرمال‌سازی شدند تا مقادیر پیکسل‌ها به بازه‌ی [0, 1] تبدیل شوند:

$$x' = \frac{x}{255.0}$$

این کار باعث بهبود سرعت یادگیری و پایداری آموزش مدل می‌شود.

مدل استفاده‌شده در این پروژه یک شبکه ساده و سبک است که از لایه‌های زیر تشکیل شده است:

#### 1. Flatten

تبدیل تصویر دوبعدی 28 × 28 به بردار یک‌بعدی

#### 2. Dense (128 neurons, ReLU)

استخراج ویژگی‌های غیرخطی از داده

#### 3. Dense (10 neurons, Softmax)

لایه خروجی برای تشخیص یکی از 10 کلاس رقم

```
17 model = keras.Sequential([
18     keras.Input(shape=(28, 28)),
19     layers.Flatten(),
20     layers.Dense(128, activation="relu"),
21     layers.Dense(10, activation="softmax")
22 ])
23
```

در مرحله آموزش، مدل با داده‌های MNIST آموزش داده شد.

برای هر epoch، داده‌ها به صورت batch‌های کوچک به مدل داده شدند تا فرآیند یادگیری به صورت مرحله‌ای انجام شود.

پارامترهای اصلی آموزش:

- تعداد epoch: 5

- اندازه batch: 32

تابع `loss` و `optimizer` نیز به صورت استاندارد در TensorFlow/Keras تعریف شد.

طبق خواسته‌ی پروژه، برای هر لایه و هر `epoch` از `thread` استفاده شد.

هدف از این کار، نمایش مفهوم پردازش موازی و هماهنگ‌سازی وظایف در سیستم عامل است.

در این پروژه:

- هر `epoch` در یک `thread` جداگانه اجرا شد
- لایه‌های شبکه نیز به صورت مستقل و مرحله‌ای مدیریت شدند
- اجرای هم‌زمان بدون هماهنگی می‌توانست باعث به‌هم‌ریختگی داده‌های مشترک شود بنابراین، `thread`ها برای تقسیم وظایف آموزشی و اجرایی استفاده شدند.

برای جلوگیری از بروز `race condition` و حفظ صحت داده‌های اشتراکی، از `Semaphore` استفاده شد.

داده‌هایی مانند `train_x` و `train_y` منابع اشتراکی هستند و اگر چند `thread` هم‌زمان به آن‌ها دسترسی پیدا کنند، ممکن است نتیجه‌ی آموزش اشتباه شود.

در نتیجه، تعداد حداقلی `semaphore` برای کنترل دسترسی به بخش‌های حساس برنامه استفاده شد تا:

- داده‌های آموزشی به صورت صحیح خوانده شوند
- محاسبات نهایی درست باقی بمانند
- از تداخل `thread`ها جلوگیری شود

به این ترتیب، هم‌زمانی در برنامه برقرار شد، اما بدون ایجاد خطا در داده‌های مشترک.

```
sem12 = Semaphore(0)
sem23 = Semaphore(0)

# Layer 1 Worker Threads
```

```
# Start Layer threads
t1 = threading.Thread(target=layer1)
t2 = threading.Thread(target=layer2)
t3 = threading.Thread(target=layer3)

t1.start()
t2.start()
t3.start()

t1.join()
t2.join()
t3.join()
```

برای راحتی کاربر، یک رابط گرافیکی با tkinter طراحی شد که قابلیت‌های زیر را دارد:

- نمایش تصویر MNIST
- نمایش نتیجه پیش‌بینی مدل
- امکان رسم رقم توسط کاربر
- دکمه‌های کنترل برای اجرای پیش‌بینی
- مشخص کردن تعداد لایه‌های اول

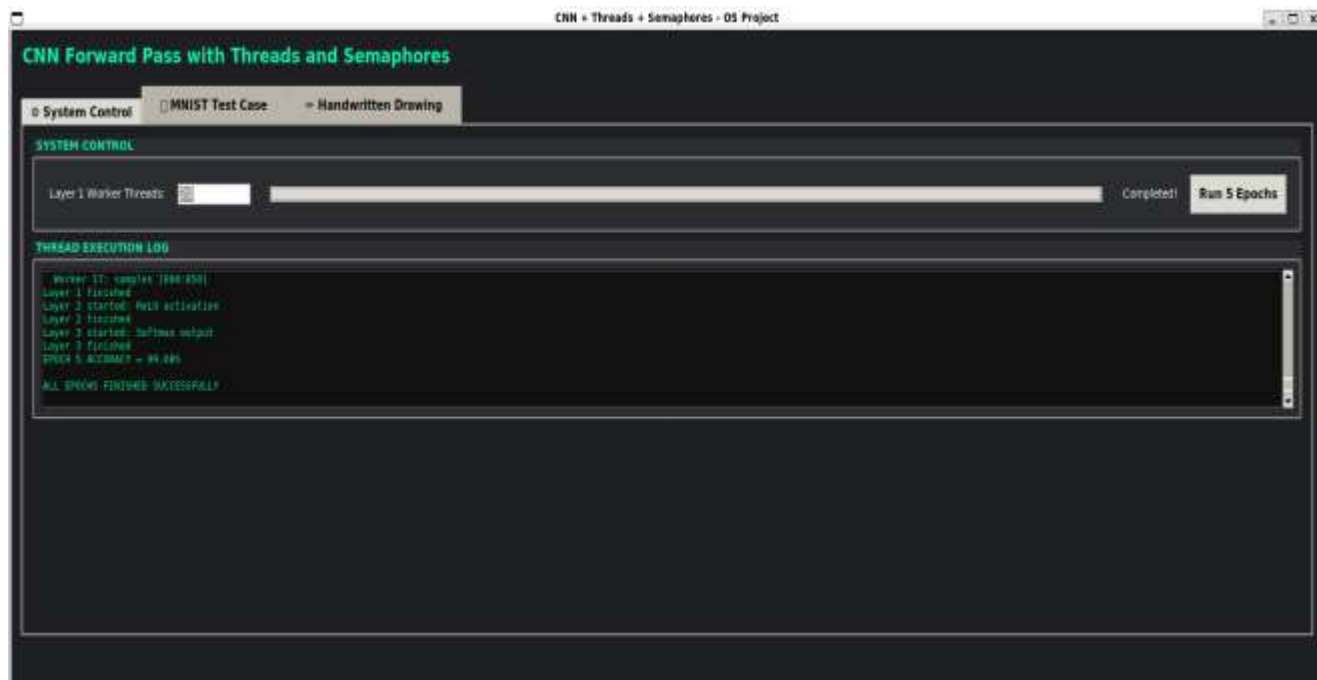
```
# Layer 1 Worker Threads
def layer1():
    log_callback(f"Layer 1 started with {num_threads} worker threads")

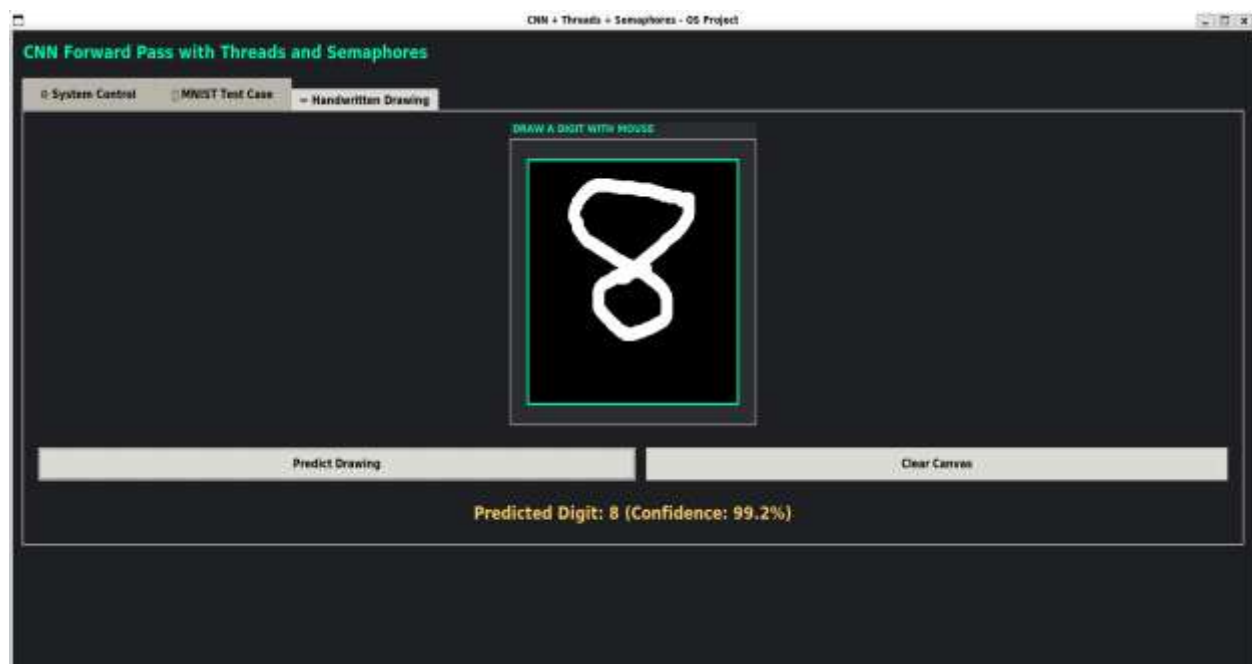
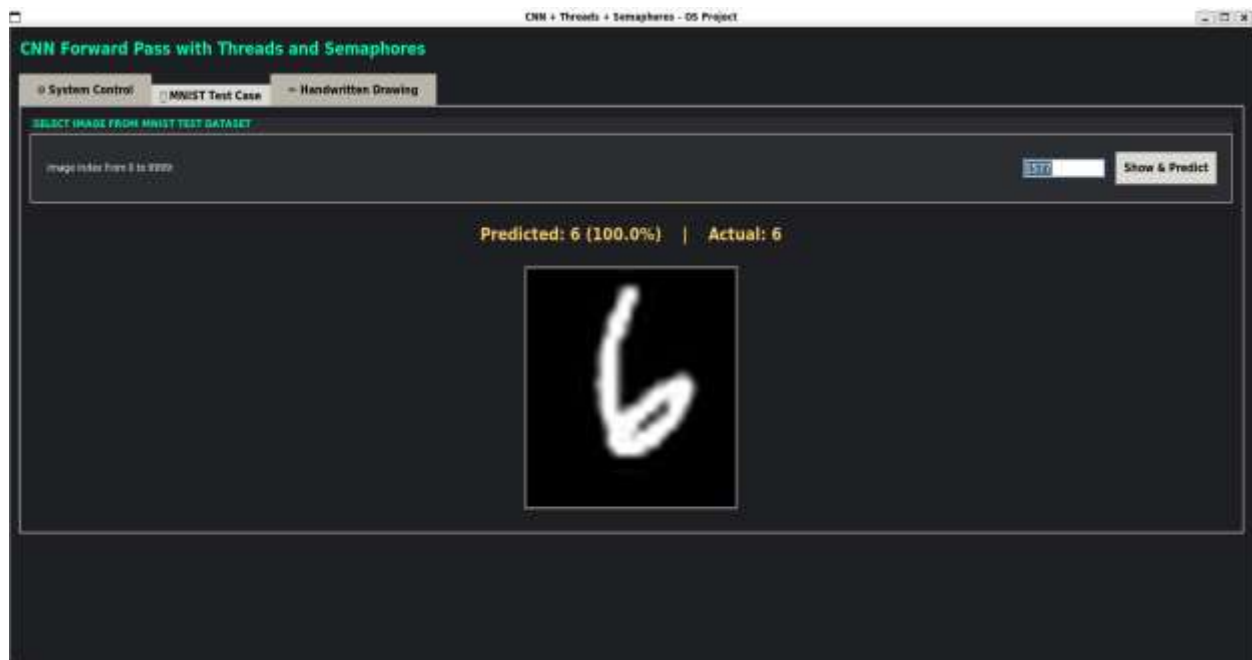
    flat = batch_x.reshape(-1, 784)
    context["flat"] = flat
    context["l1"] = np.zeros((flat.shape[0], 128))

    chunk = int(np.ceil(flat.shape[0] / num_threads))
    workers = []
```

پروژه در محیط WSL / Ubuntu اجرا شد.

```
nouri@DESKTOP-4HROFCV: / X + v
nouri@DESKTOP-4HROFCV:/mnt/d/university/term5/os/project/CNN-project$ source venv/bin/activate
(venv) nouri@DESKTOP-4HROFCV:/mnt/d/university/term5/os/project/CNN-project$ python cnn_semaphore_gui.py
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1783082507.625737    1191 port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results
due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable 'TF_ENABLE_O
NEDNN_OPTS=0'.
I0000 00:00:1783082507.632533    1191 cudart_stub.cc:31] Could not find cuda drivers on your machine, GPU will not be used.
I0000 00:00:1783082508.540743    1191 cpu_feature_guard.cc:227] This TensorFlow binary is optimized to use available CPU instructions
in performance-critical operations.
To enable the following instructions: AVX2 AVX_VNNI FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1783082531.327891    1191 port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results
due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable 'TF_ENABLE_O
NEDNN_OPTS=0'.
I0000 00:00:1783082531.345224    1191 cudart_stub.cc:31] Could not find cuda drivers on your machine, GPU will not be used.
E0000 00:00:1783082542.062563    1191 cuda_platform.cc:52] failed call to cuInit: INTERNAL: CUDA error: Failed call to cuInit: UNKNOWN
N ERROR (303)
Epoch 1/5
1875/1875 ————— 5s 2ns/step - accuracy: 0.9262 - loss: 0.2689
Epoch 2/5
1875/1875 ————— 4s 2ns/step - accuracy: 0.9651 - loss: 0.1164
Epoch 3/5
1875/1875 ————— 4s 2ns/step - accuracy: 0.9758 - loss: 0.0798
Epoch 4/5
1875/1875 ————— 4s 2ns/step - accuracy: 0.9822 - loss: 0.0595
Epoch 5/5
1875/1875 ————— 4s 2ns/step - accuracy: 0.9855 - loss: 0.0466
```





### تحلیل فنی همزمانی و استفاده از Semaphore

در این پروژه، چالش اصلی هماهنگ‌سازی سه بخش اصلی یعنی آماده‌سازی داده‌ها (Epoch/Data Loader)، محاسبات لایه‌ها (Layer Processing) و بخش پیش‌بینی/خروجی نهایی بود. برای حل این چالش از دو سمافور استفاده شده است:

1. **سمافور اول (sem12):** این سمافور وظیفه دارد مطمئن شود لایه دوم (پردازش ویژگی‌ها) کار خود را شروع نمی‌کند مگر اینکه لایه اول (بارگذاری و Flatten کردن داده‌ها برای Epoch جاری) داده‌ها را آماده کرده باشد.
2. **سمافور دوم (sem23):** این سمافور تضمین می‌کند که بخش خروجی و محاسبات نهایی (Dense 10) اجرا نمی‌شود تا زمانی که محاسبات لایه میانی (Dense 128) به طور کامل روی داده‌های اشتراکی پایان یافته باشد.

**روند هم‌زمانی (Synchronization Flow):**

- **Thread 1 (ورودی):** داده را آماده می‌کند → سیگنال می‌دهد (sem12.release)
- **Thread 2 (لایه میانی):** منتظر می‌ماند (sem12.acquire) → پردازش می‌کند → سیگنال می‌دهد (sem23.release)
- **Thread 3 (خروجی):** منتظر می‌ماند (sem23.acquire) → خروجی را ثبت می‌کند.